

程序设计语言初步

-数据部分：类型、常量、变量

软件与理论中心

张艳梅



计算机中的数据表示

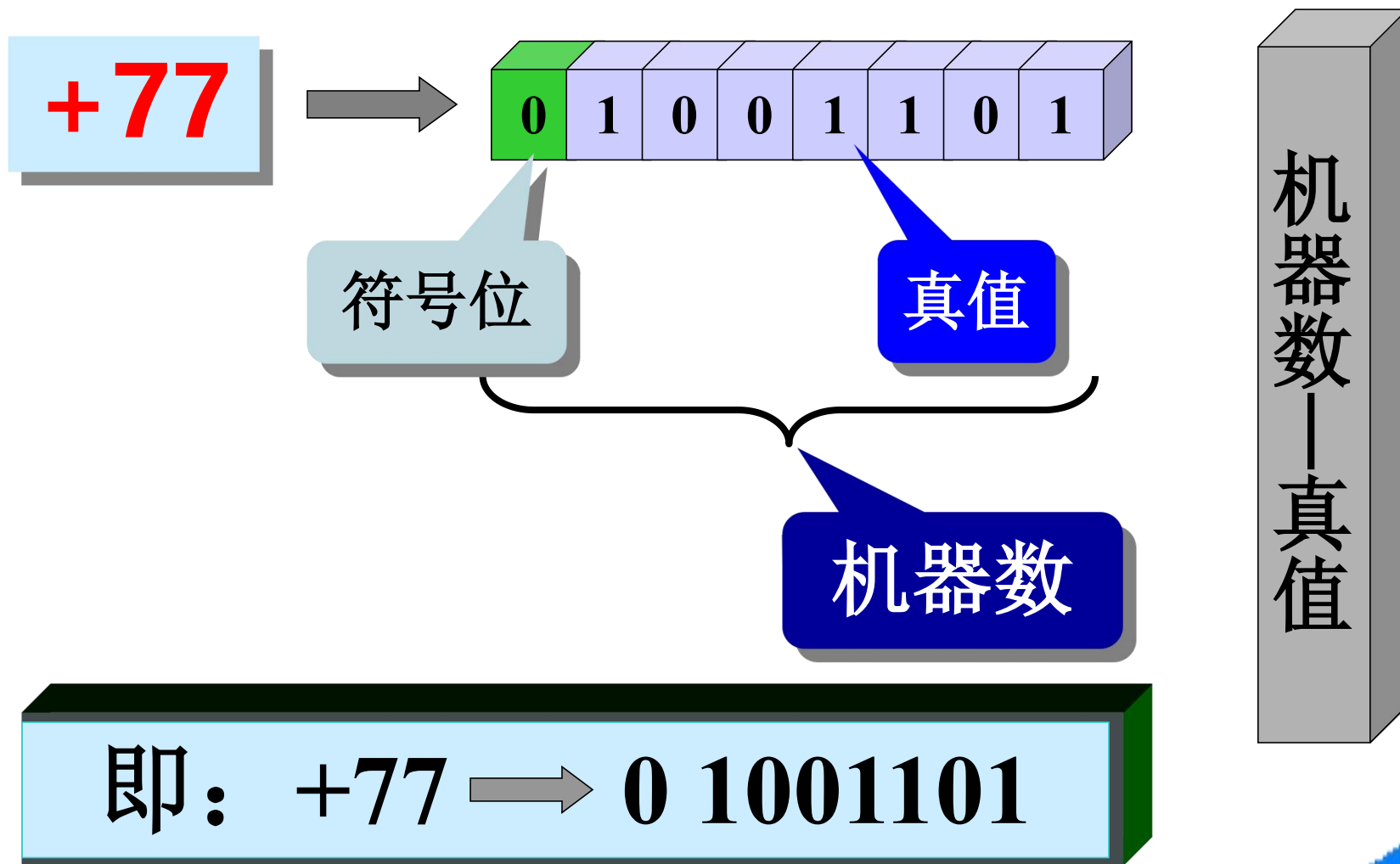
- 计算机的基本功能是对数据进行运算和加工处理。
- 数据有两种,一种是数值数据,如3.1416、-2.71828……,另一种是非数值数据(信息),如A, b, +, = ……。
- 无论哪一种数据在计算机中都是用二进制数码表示的。数值处理采用二进制运算;非数值处理采用二进制编码,它们具有运算简单、电路实现方便、成本低廉等优点。



计算机存储单元：位、字节、字

- 最小的存储单元是**位**（bit），可以存储0或1（或者说，位用于设置“开”或“关”）。
- **字节**（byte）是常用的计算机存储单位。对于几乎所有的机器，1字节均为8位。通过二进制编码，便可表示0~255的整数或一组字符。
- **字**（word）是设计计算机时运算单元给定的整数运算的数据通路的宽度。因此，计算机中每个数据的二进制位数都是固定长度的。
- $1\text{Kb} = 2^{10} = 1024 \text{ byte}$
- $1\text{Mb} = 1024 \text{ Kb}$
- $1\text{Gb} = 1024 \text{ Mb}$

计算机中数据的表示方法



一、机器数

- 计算机中数的正负的表示：用二进制数的最高位来表示符号。如果最高位为1，则是负数；为0，则是正数。
- 机器数：增加了符号位的二进制数称为机器数，它的数值称为机器数的真值。
- 机器数的编码格式：补码表示法。

机器数的编码格式(1)

例1：两个正数的机器数相加：43 + 43

$$X = (+43)_{10}, [X]_{\text{原码}} = 0\ 0101011$$

$$Y = (+43)_{10}, [Y]_{\text{原码}} = 0\ 0101011$$

$X + Y$:

$$\begin{array}{r} 0\ 0101011 \\ +\ 0\ 0101011 \\ \hline 0\ 1010110 \end{array} = 86 \quad \text{结果正确}$$

机器数的编码格式(2)

例2：正数和负数的原码相加： $43 + (-43)$

$$X = (+43)_{10}, [X]_{\text{原码}} = 0\ 0101011$$

$$Y = (-43)_{10}, [Y]_{\text{原码}} = 1\ 0101011$$

$X + Y$:

$$\begin{array}{r} 0\ 0101011 \\ +\ 1\ 0101011 \\ \hline 1\ 1010110 \end{array}$$

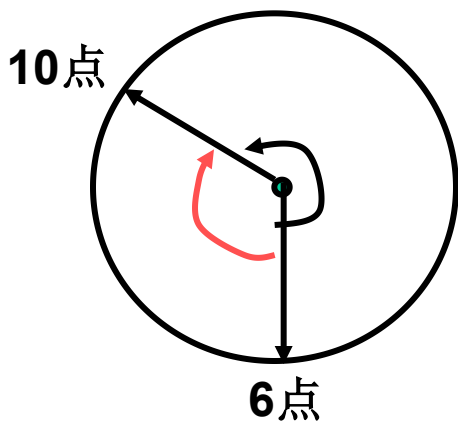
$= -86$ 结果不正确！



机器数的编码格式(3)-补码的由来

变减为加原理——时钟的例子：

时钟的6点减去8个小时，相当于将时钟逆时针调8个小时，结果是10点；时钟是圆形的，逆时针调8个小时 = 顺时针调4个小时。逆时针调是减法，顺时针调是加法。



结论：在时钟的12进制上，

$$6 - 8 = 6 + 4$$

而-8和+4在模12上互为补数。

因此 $6 - 8 = 6 + (-8 \text{ 的补数})$

从而实现了变减为加

机器数的编码格式(4)-补码的由来

➤ 为了把减法变成加法运算，设计了补码。

➤ 补数的概念：

n 位二进制数能表示的数字，都小于 2^n 。在计算结果不溢出的前提下，这些数字称为模 2^n 的数据。

设 x 是模 2^n 的数，则 $|x\text{的补数}| = 2^n - |x|$ ， x 的补数的正负和 x 正好相反。

例如： x 是模12的数， $x = -7$ ，则 x 的补数是5；

x 是模128的数， $x = -15$ ，则 x 的补数是113。

机器数的编码格式(5)-如何得到补码

- 正数的补码表示与原码相同。
- 负数的补码，是负数的原码除符号位不变之外，按位取反，然后在最低位加1得到的结果。

例如： $X = (-43)_{10}$

$[X]_{\text{原码}} = 1\ 0101011$

$[X]_{\text{反码}} = 1\ 1010100$

$[X]_{\text{补码}} = 1\ 1010101$

例3： $43 + (-43)$

$0\ 0101011$

$+ 1\ 1010101$

$=$

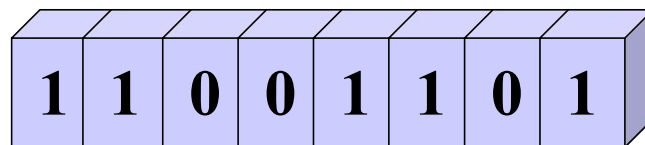
计算机中数据的表示方法

带符号的机器数

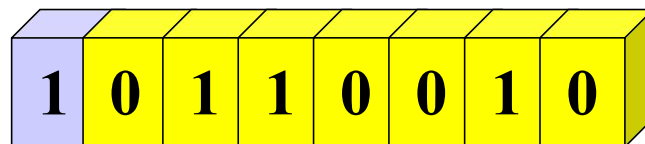
-77



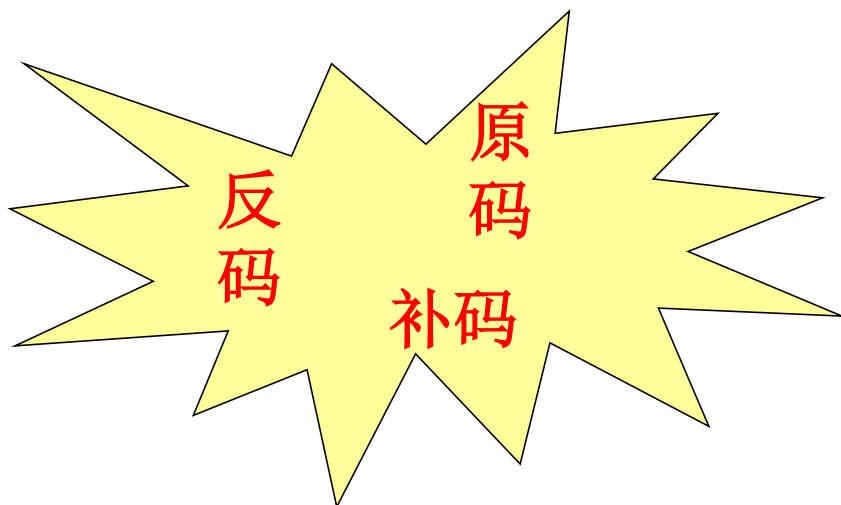
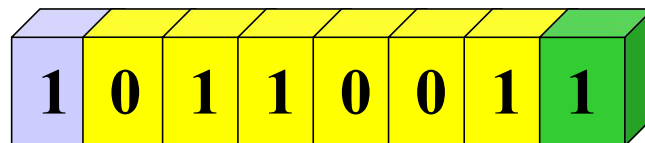
原码



反码



补码



字符型

- 基础字符数据包括26*2个字母（大小写）+10个数字+其它显示符号，总共95种；还有33种控制符号，总共128种符号；
- 字符型数据在内存中的存储：实际存放的是一个整数值。多数计算机系统采用ASCII（*American Standard Code for Information Interchange*）标准编码模式来对字符进行编码。每个字符占用8位内存（1个字节）；

ASCII表

L \ H	0000	0001	0010	0011	0100	0101	0110	0111
0000	NUL	DLE	SP	0	@	P	'	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	,	7	G	W	g	w
1000	BS	CAN	)	8	H	X	h	x
1001	HT	EM	(	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	'	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

汉字信息的数字化





浮点数

- 计算机中的小数，采用浮点数表示法。浮点数是指小数点的位置是可以浮动的。
- E计数法：一个实数总可以表示成一个乘幂和一个纯小数之积。例如：

$$123.45 = 10^3 \times (0.12345), \text{记作 } 0.12345\text{e}3$$

$$-0.0034574 = 10^{-2} \times (-0.34574), \text{记作 } -0.34574\text{e}-2$$

- 计算机中的小数表示，使用二进制形式的e计数法。乘幂中的指数部分我们称其为“阶码”（这是一个整数）；纯小数部分我们称其为“尾数”。尾数和阶码都有正负之分，且都有固定位数。

$$\text{例如: } 1001.011 = 2^{4(100)} \times (0.1001011)$$

$$-0.0010101 = 2^{-2(10)} \times (-0.10101)$$

数据是计算机处理的对象。但计算机表示数据的方式只有整数类型和浮点数类型两大类。

数值类的数据依据其本身的特点可以归为不同的类：**整数、分数、无理数、实数、复数**等。

非数值类的文字、声音、图片等信息都可以数字化为整数编码。

问题： **7.0 、 $\frac{1}{3}$ 、 $\sqrt[2]{2}$ 、 r^2 、 π 、 $\sin \theta$** 该如何设计数据类型？

- 数据类型包括两层含义：定义了值的集合（属于该类型的数据能够**取值的范围**）以及能应用于这些数值上的操作集合（**数据操作**）。
- 整数集合，支持编码转换，以及整数的加、减、乘和整数除法运算。
- 浮点数集合，支持算术运算、数学函数运算。但是浮点数只能近似表示，例 $1/3$ 只能表示为有限位的 $0.333\dots$

切记：整数和浮点数在计算机中是两种不同表示方式，数据类型必须区分开。

- 当一个整数被一个正整数除, 可得一个商和一个余数. 传统上这被叫做“除法算法”, 但其实这是一个定理.

除法算法: 若 a 是整数, d 是正整数, 则存在唯一的一对整数 q 和 r , $0 \leq r < d$, 使得 $a = dq + r$ (数学已证明).

- d 称为除数.
- a 称为被除数.
- q 称为商.
- r 称为余数.

函数 `div(/)` 和 `mod(%)` 的定义:

$$q = a / d$$

$$r = a \% d$$

例:

- 计算101除11的商和余数?
- 计算-11除3的商和余数?

- 常量是指在程序执行过程中不能改变的数据。常量按表示形式分为：文字常量、命名常量、符号常量。
- 文字常量**是指在程序中直接出现的数据。例：

15

整型常量

15.4

双精度浮点型常量

15.4f

单精度浮点型常量

'A'

字符型常量（英文单引号）

"Hello world!" 字符串型常量（英文双引号）

- 若整数常量以0开头，C编译器会认为该常量是八进制数。例040表示十进制的32。
- 若整数常量以0x开头，C编译器会认为该常量是十六进制数。例0xFF即十进制的255。

- 符号常量：程序代码中预先统一设定的文字常量，并以自定义的标识符表示。类似速记中的速记符号。

符号常量的使用格式：

`#define` 常量名 文字常量或表达式

例：

```
#define N 100
int main(void){
    printf("%d\n",2*N);
}
```

命名常量：被保护的变量

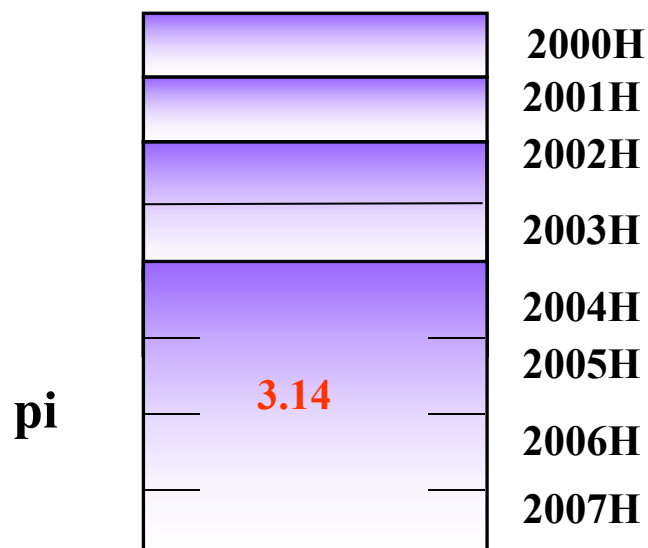
◆不能改写的变量(命名常量)

有些变量一旦声明并初始化, 便不允许程序去改变该存储空间中的数据。

C语言中定义一个不能改写的变量:

```
const float pi = 3.14;
```

存储空间



- `printf()`函数将指定格式的值打印在显示器上。

- 语法:

```
int printf(const char *format, ...);
```

- 转换说明`format`是由双引号表示的字符串，结构如下:

%[flags][width][.precision][modifier]type

`flags`指定打印格式，`width`指定数值位数，`precision`指定小数点后位数，`modifier`可对类型`type`添加`h`短、`l`长修饰。

- 最简单的格式串由一个百分号（%）和一个类型字符组成，如`%d`或`%f`，`%d`表示按整数类型打印，`%f`表示按浮点数打印，`%g`表示忽略小数末尾的0按最短长度打印

```
#include<stdio.h>
int main(void){
    printf("%d\n",12);
    printf("%#x\n",0x3);//16进制前缀打印
    printf("%c\n",'c');
    printf("%c\n",'\\041');
    printf("%e\n",2.34e07);
    printf("%f\n",7.0);
}
```

- ◆ 计算机中的变量，指在程序执行过程中可能改变或被赋值的数据。
- ◆ 变量的命名规则，遵从标识符规则 (Identifier)。
- ◆ C语言规定：标识符由英语大写字母A到Z、小写字母a到z、数字0到9和下划线组成，但第一个字符必须是字母或下划线。
- ◆ 思考：以下哪些是合法的C语言标识符
`age, _class_no, name1, name1%, 2name, #age`

- 使用变量前，必须先声明变量的数据类型和名称。

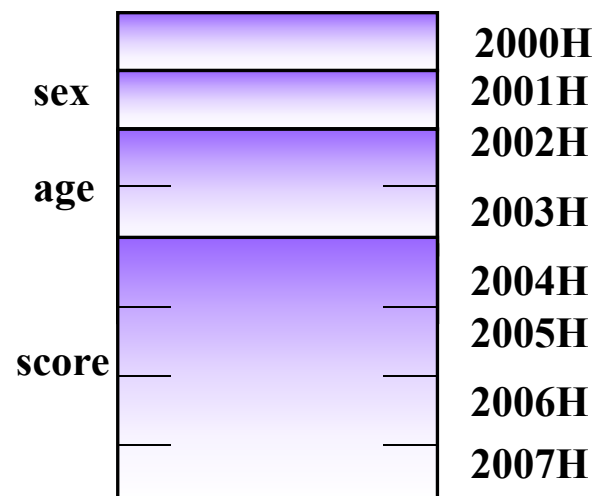
- C变量声明格式：

数据类型 变量名1[,变量名2, …];

变量声明将引起内存空间的分配, 保证变量名对应内存中的一片存储单元（存储单元个数取决于变量的数据类型）。

存储空间

```
char sex;  
int age;  
float score;
```



- 变量声明时，可以给变量一个初始值。

- C变量初始化格式：

数据类型 变量名1=值1[,变量名2, …];

程序算法中涉及判断逻辑，或者暂存中间结果的变量，一定要初始化，因为分配的内存空间不会自动初始化。

```
int hogs = 21;  
int cows = 32, goats = 14;  
int dogs, cats = 94; /*有效，但是这种格式很糟糕 */
```

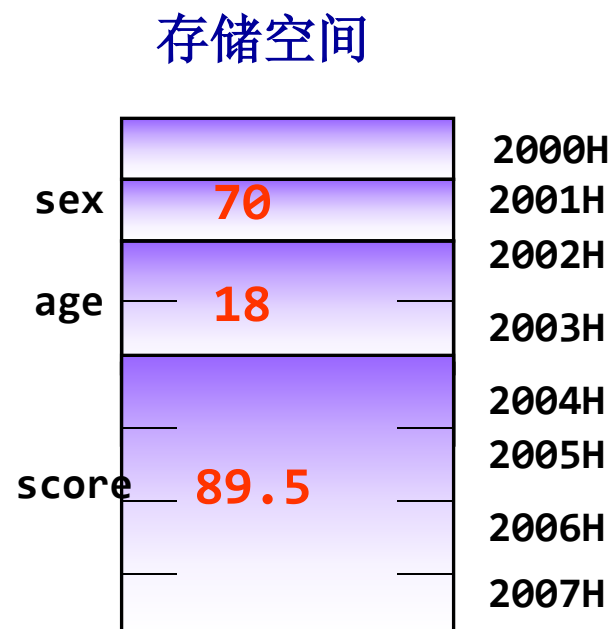
- 变量赋值：把一个值写入已声明变量代表的存储空间。
- C语言变量赋值格式：
变量名=表达式

例：

`sex='F';`

`age=18;`

`score=89.5;`



C语言的整数类型关键字

数据类型关键字	含义
int	整数
short	短整数，节省空间
long	长整数，位数不比int小
long long	较大整数，至少 64 位
unsigned	无符号整数（非负整数）
char	字母、字符，ASCII字符集和扩展的各种自然语言字符集

- sizeof关键字的使用格式:

`size_t sizeof (type或expression)`

- 关键字**sizeof**返回给定的数据类型或表达式结果值所占内存的字节数.

`/* sizeof.c—测试各个数据类型的存储长度*/`

```
#include <stdio.h>
```

```
int main(void){
```

```
    printf("int length=%d\n",sizeof(int));
```

```
    printf("short length=%d\n",sizeof(short));
```

```
    printf("long length=%d\n",sizeof(long));
```

```
    printf("long long length=%d\n",sizeof(long long));
```

```
    printf("unsigned length=%d\n",sizeof(unsigned));
```

```
    printf("char length=%d\n",sizeof(char));
```

```
    printf("float length=%d\n",sizeof(float));
```

```
    printf("double length=%d\n",sizeof(double));
```

```
/* toobig. c--超出系统允许的最大int值*/  
#include <stdio.h>  
int main(void)  
{  
    int i = 2147483647;  
    unsigned int j = 4294967295;  
    printf("%d %d %d\n", i, i+1, i+2);  
    printf("%u %u %u\n", j, j+1, j+2);  
    return 0;  
}
```

printf()数据类型不匹配的测试

```
/* print2.c--更多printf()的特性*/  
#include <stdio.h>  
int main(void)  
{  
    unsigned int un = 3000000000; /*int为32位和short为16位的系  
统*/  
    short end = 200;  
    long big = 65537;  
    long long verybig = 12345678908642;  
    printf("un = %u and not %d\n", un, un);  
    printf("end = %hd and %d\n", end, end);  
    printf("big = %ld and not %hd\n", big, big);  
    printf("verybig= %lld and not %ld\n", verybig, verybig);  
    return 0;  
}
```

- `scanf()`函数根据格式串限定的数据类型，接受键盘输入的变量值。读取基本类型的变量值，需要在变量名前加上一个`&`。
- 语法: `int scanf(const char *format, ...);`
- 输入格式串`format`是由双引号表示的字符串，结构如下: `%[*][width][modifier]type`
 - *表示丢弃该数，`width`限定数值读取最大位数，`modifier`可对类型`type`添加`h`短、`l`长修饰。
- 输入格式串最重要的是指定数据类型，最简单的形式由百分号（`%`）和一个类型字符组成。

scanf函数格式串说明

- scanf()函数的输入格式串format是由双引号表示的字符串，结构如下：

%[*][width][modifier]type

- %d表示10进制整数，%u表示无符号整数，%lld表示long long int类型。
- %f表示单精度浮点数，%lf表示双精度浮点数，%e表示接受一个e计数格式的小数，
- %c表示单个字符。%s表示多个字符构成的字符串。
- %o表示8进制无符号整数，%x表示16进制无符号整数。
- 除了%c，其他转换说明都会自动跳过待输入值前面所有的空白。因此，scanf("%d%d",&n,&m)与scanf("%d %d",&n,&m)的作用相同。

```
#include<stdio.h>
int main(void)
{
    int a,b;
    scanf("%d%d",&a,&b);
    printf("%d+%d=%d\n",a,b,a+b);
    return 0;
}
```

C程序示例：字符的存储

```
/*认识字符在计算机中的存储和显示*/  
#include<stdio.h>  
int main(void)  
{  
    char ch;  
    scanf("%c",&ch);  
    printf("char:%c,int:%d \n",ch,ch);  
    return 0;  
}
```

输入'A',结果char:A,int:65

C语言的浮点数类型关键字

数据类型关键字	含义
float	浮点数，单精度小数
double	双精度浮点数

```
/*float.c - 测试浮点数的精度 */
#include <stdio.h>
int main(void)
{
    printf("approximate float number exam.\n");
    float data;    /*变量定义*/

    printf("Enter a real number:");
    scanf("%f",&data); /*从键盘读取float小数*/
    printf("stored number is : %.8f\n",data);
    /*打印实际存入计算机的小数*/
    return 0;
}
```

5. 测试和调试程序

测试数据1: 0.32

测试数据2: 3.99

测试数据3:

3.141592653589793238462643383279502884197

169399375105820974944592307816406

测试数据4: 0.000000005

测试数据5: -1234567890.0012345

精度：计算机中的小数是有限位的

IEEE754规定，*float*单精度浮点数字长32位，分配如下：

正负号	尾数	指数
占1位	占23位	占8位
0正,1负	小数位, 决定精度	整数, 决定范围

尾数最大值 $1.111\cdots$ (23个1), 约等于2; 指数8位最大是127; 因此32位float最大值为 $2 \times 2^{127} = 3.4 \times 10^{38}$, 负值最小同理为 $-2 \times 2^{127} = -3.4 \times 10^{38}$ 。可表示十进制的6或7位有效数字。

Double双精度浮点数字长64位。正负号1位, 尾数52位, 指数11位。可表示十进制的15或16位有效数字。

十进制小数转换为二进制小数

乘基取整法：“乘基取整，先整为高(位)，后整为低(位)”

例：(0.625)₁₀ = (0.101)₂

	0.625	整数
×	2	
	1.25	1
	0.25	
×	2	
	0.5	0
×	2	
	1.0	1

十进制小数并不是都能够用有限位的二进制数精确地表示, 这时应根据精度要求转换到一定的位数为止, 作为其近似值.

例：(0.32)₁₀ = (0.0101...)₂

	0.32	整数
×	2	
	0.64	0
×	2	
	1.28	1
	0.28	
×	2	
	0.56	0
×	2	
	1.12	1
	...	...


```
/* badcount.c --参数错误的情况 */
#include<stdio.h>
int main(void)
{
    int n = 4;
    int m = 5;
    float f = 7.0f;
    float g = 8.0f;
    printf("%d\n", n, m);/*参数太多*/
    printf("%d %d %d\n",n); /*参数太少*/
    printf("%d %d\n", f, g); /*值的类型不匹配*/
    return 0;
}
```

- 计算机中的浮点数和整数在本质上不同，其存储方式和运算过程有很大区别。
- 即使两个32位存储单元存储的位组合完全相同，但是一个解释为**float**类型，另一个解释为**long**类型，这两个相同的位组合表示的值也完全不同。例如，在PC中，假设一个位组合表示**float**类型的数256.0，如果将其解释为**long**类型，得到的值是113246208。
- C语言允许编写混合数据类型的表达式，但是会进行自动类型转换，以便在实际运算时统一使用一种类型。

- **问题描述：**编写一程序，按照顺序分别输入某学生的性别、年龄和身高(以米为单位)，你的任务是将三者按照规定的格式输出。性别要用一个字符型(char型)变量存储，年龄要用一个整型(int 型)变量存储，身高要用一个单精度浮点型(float 型)变量存储。输入时三者各占一行。
- **输入与输出要求：**首先输入的是一个字符‘M’或‘F’ 代表性别；然后输入的是一个整数，代表年龄；最后输入的是一个浮点数，代表身高。你要输出的语句形如：“The sex is M, the age is 18, and the height is 1.660000.”，这句话要占一行。
- **程序运行效果：**

Sample 1:

- Please input sex: M✓
- Please input age: 18✓
- Please input height:1.66 ✓
- The sex is M, the age is 18, and the height is 1.660000.

